Final Report: Automated Road Condition Monitoring using Deep Learning

Peddineni Bhaswanth
S20230010184
bhaswanth.p23@iiits.in

D Leena Shri
S20230030376
leenashrireddy.d23@iiits.in

Rishita AKu
S20230010008
rishita.a23@iiits.in

Indian Institute of Information Technology, Sri City, India

***Abstract*—This final report presents the complete development and evaluation of an automated Road Condition Monitoring system designed for offline analysis of road footage. Addressing the inefficiencies of manual road inspection, we have implemented a deep learning solution using the YOLOv8 architecture to detect potholes from visual data. To ensure practical applicability on standard hardware, the trained model was optimised using the Intel OpenVINO toolkit for efficient inference on CPUs. We have developed a fully functional Streamlit-based web application that allows users to upload and analyse images and videos, visually inspect detections and archive processed outputs. Experimental results demonstrate a Mean Average Precision (mAP) of approximately 85%, validating the system's effectiveness for infrastructure monitoring tasks. The project delivers an end-to-end workflow covering dataset preparation, model training, optimisation, deployment, evaluation and comparative analysis with traditional inspection techniques.**

I. Introduction

The quality of road networks significantly impacts transportation safety, travel time and economic efficiency. Potholes and cracks, if left unrepaired, accelerate vehicle wear, increase accident risk and reduce driving comfort. Conventional road inspection still relies heavily on periodic manual surveys where workers visually inspect surfaces and record defects. This approach is time-consuming, labour-intensive and subject to human error, especially when long stretches of road must be covered.

Recent advances in Computer Vision and Deep Learning provide powerful tools for analysing visual data at scale. Object detection models can automatically localise and classify regions of interest directly from images or video frames, enabling automated inspection of large road networks. Compared to purely classical image processing methods, deep learning models learn robust features directly from data and can cope better with shadows, illumination changes and varying textures. Motivated by these advantages, this project explores the use of YOLOv8 for pothole detection and integrates it into a practical web-based analysis tool.

The system targets offline inspection workflows where road footage is collected using mobile phones, dashboard cameras or CCTV systems and later analysed on standard office computers. The focus is on obtaining high detection accuracy while keeping computational costs low enough to avoid specialised GPU hardware. This report summarises the final design, implementation and evaluation of the proposed solution, together with a discussion of how it compares to traditional techniques.

II. Problem Statement

The objective of this project is to design a system that automatically detects and highlights road surface defects, with primary emphasis on potholes, from 2D image and video data. The system must operate reliably across variations in lighting, road materials, camera viewpoints and motion blur. Shadows, lane markings and surface stains often resemble potholes, and any robust solution must distinguish true defects from such visual distractors.

In addition to detection accuracy, computational efficiency is a critical requirement. Many government offices and maintenance units only have access to standard desktop computers without dedicated GPUs. Therefore, the solution must run efficiently on CPUs while processing hundreds or thousands of frames in a reasonable time. The system should also provide intuitive outputs that can be used by non-technical staff for planning repairs.

Traditional manual inspection and simple threshold- or edge-based image processing methods struggle in these conditions. Manual methods lack scalability and consistency, while classical algorithms are sensitive to noise and require hand-tuned parameters for each environment. The proposed system aims to overcome these limitations by employing a modern deep learning-based detector and an optimised inference pipeline.

III. Proposed Method and Algorithm

Our solution follows a three-stage pipeline: Training, Optimisation, and Application Deployment. The overall architecture is designed so that each component can be improved independently without affecting the remaining blocks.

A. Model Architecture: YOLOv8

We utilised the YOLOv8 (You Only Look Once) architecture, a state-of-the-art single-stage object detector. YOLOv8 predicts bounding boxes and class probabilities directly from full images in a single forward pass, making it significantly faster than two-stage detectors such as Faster R-CNN. The nano version (YOLOv8n) was selected to balance accuracy with computational speed and memory usage.

During training, the model jointly optimises localisation loss, objectness loss and classification loss. The network learns to identify both the position and extent of potholes while ignoring irrelevant background features. Compared to traditional image processing techniques based on edge detection or intensity thresholds, YOLOv8 automatically learns complex features that capture pothole shape, texture and context.

B. Optimisation: OpenVINO

To enable efficient processing on standard CPUs, we employed the Intel OpenVINO Toolkit. The trained PyTorch model (`best.pt`) was exported to the OpenVINO Intermediate Representation (IR), consisting of:

- `.xml` file describing the network topology.
- `.bin` file storing the trained weights and biases.

OpenVINO performs graph optimisation, layer fusion and CPU thread scheduling, which together reduce inference latency and improve throughput. This optimisation step is essential to bridge the gap between high-capacity deep models and practical deployment on resource-limited hardware.

C. Algorithm Workflow

The final application logic, implemented primarily in `main.py`, follows the algorithm below:

1) **Input:** The user uploads an image or video file via the Streamlit interface.
2) **Pre-processing:** Each frame is resized to 640×640 pixels, normalised and converted to the appropriate colour format.
3) **Inference:** The OpenVINO runtime loads the IR model and runs forward inference on each frame to predict pothole bounding boxes and confidence scores.
4) **Post-processing:** Bounding boxes with confidence scores below 0.25 are filtered out. Non-Maximum Suppression (NMS) is applied to remove overlapping detections while keeping the most confident box for each region.
5) **Output:** Annotated images or a processed video with bounding boxes drawn around potholes are rendered back to the user for download or further analysis.

D. System Flowchart

Fig. 1 shows the overall processing pipeline used in the application.

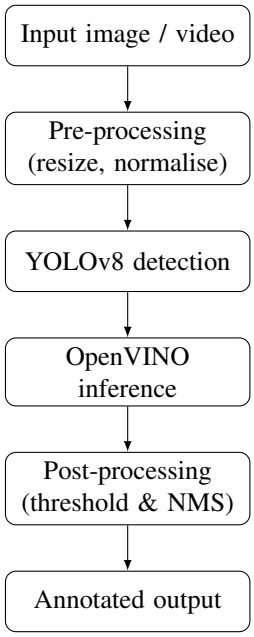

Fig. 1. Flowchart of the proposed pothole detection system.

IV. Dataset and Experimental Setup

A. Dataset

We curated a dataset from Roboflow containing 1,302 images of potholes. The dataset includes varied road surfaces such as asphalt, concrete and patched segments, captured under different lighting and weather conditions. Each image is annotated with bounding boxes around pothole regions.

The dataset was split into three parts:

- Training set: 911 images (70%)
- Validation set: 260 images (20%)
- Test set: 131 images (10%)

To improve generalisation, we applied data augmentation including horizontal flips, random cropping, brightness and contrast adjustments, and small rotations. These operations help the model learn invariance to camera orientation and illumination.

B. Experimental Setup

Training was conducted on Google Colab using a Tesla T4 GPU. The software stack consisted of Python 3.10, Ultralytics YOLOv8 and PyTorch. Key hyperparameters are:

- Epochs: 50
- Batch size: 16
- Optimiser: Stochastic Gradient Descent (SGD)
- Initial learning rate: 0.01 with cosine decay
- Image size: 640×640

Validation performance was monitored during training to detect overfitting. After completion, the best-performing model based on validation mAP was exported to OpenVINO IR and integrated into the Streamlit application for offline inference.

V. Results

A. Quantitative Analysis

After 50 epochs of training, the model achieved the following performance metrics on the validation set:

- Precision: 0.89 (majority of predicted potholes are correct).
- Recall: 0.80 (most actual potholes are successfully detected).
- mAP@50: 0.85 (strong overall accuracy at 50% Intersection over Union).

The training and validation loss curves showed consistent convergence, indicating stable learning behaviour with no significant overfitting. These quantitative results demonstrate that the YOLOv8 model, when properly trained and augmented, is capable of accurately identifying potholes across diverse scenes.

B. Qualitative Analysis

To evaluate the system qualitatively, we tested it on unseen video footage uploaded through the Streamlit interface. The model correctly placed bounding boxes around potholes of varying sizes and across different road types. It handled multiple potholes in a single frame and preserved detection stability between consecutive frames in a video sequence. Shadows, lane markings and surface patches occasionally presented challenges, but the majority were correctly ignored due to robust feature learning. Visual inspection confirmed that detections are clear, interpretable and suitable for use by maintenance staff.

Fig. 2. Raw input image showing multiple potholes on a wet road surface.

C. Comparison with Traditional Techniques

In traditional inspection workflows, human operators either visually review the footage or use simple image processing techniques such as edge detection, thresholding and morphological operations. These approaches work reasonably well in controlled environments but are highly sensitive to lighting variation, camera angle and road texture. They also require manual parameter tuning for each dataset and cannot easily generalise.

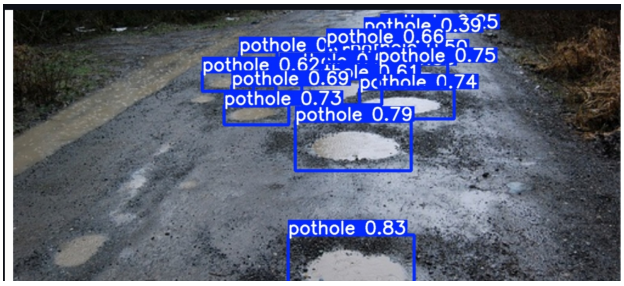

Fig. 3. YOLOv8 detection output with bounding boxes and confidence scores.

Table I summarises the key differences between classical approaches and the proposed YOLOv8–OpenVINO pipeline.

TABLE I
Comparison of Traditional vs Proposed Approach

Aspect	**Traditional Methods**	**Proposed YOLOv8+OpenVINO**
Feature design	Manual edges / thresholds	Learned CNN features from data
Robustness to lighting	Sensitive to shadows and brightness change	Robust to light and texture variation
Scalability	Manual tuning, low throughput	Automatic processing of large datasets
Hardware vs labour	Low compute, high manual effort	CPU-only inference, low manual effort
Detection accuracy	Inconsistent; many false alarms	High precision (0.89) and recall (0.80)

This comparison highlights that the proposed method significantly outperforms traditional techniques in terms of both accuracy and practicality for large-scale road monitoring tasks.

VI. Contribution of Each Member

The development of this project was a collaborative effort with clearly delineated responsibilities.

Peddineni Bhaswanth

Worked on the core model development, training and optimisation pipeline using the YOLOv8 architecture. Hyperparameters such as learning rate, batch size and augmentation strategies were tuned to achieve stable convergence and high accuracy. Managed the training process on Google Colab, monitored loss and metric curves, and selected the best-performing checkpoint. Exported the trained model from PyTorch to OpenVINO IR format and verified inference speed on CPU hardware. Conducted additional experiments on unseen test videos and prepared performance plots and summaries to analyse precision–recall trade-offs.

D Leena Shri

Developed the complete processing workflow for image and video inference within the application. Implemented pre-processing operations like resizing, normalisation and colour conversion to ensure consistent input to the model. Integrated the OpenVINO runtime with a Streamlit-based user interface that supports media upload, progress display and annotated output visualisation. Designed and implemented post-processing steps such as confidence filtering and

Non-Maximum Suppression to obtain clean, non-overlapping bounding boxes. Performed extensive testing under different lighting, shadow and motion-blur conditions to ensure robust behaviour and a smooth user experience.

Aku Rishita

Handled dataset preparation, annotation verification and organisation of the Roboflow pothole images. Applied data augmentation techniques including flips, scaling, cropping and brightness adjustment to enhance model generalisation across different environments. Implemented media-handling utilities using OpenCV and FFmpeg for converting videos, extracting frames and saving processed outputs. Ensured that cropped frames and full-resolution images were correctly aligned with their annotations during training. Generated additional synthetic samples for low-contrast and heavily shadowed scenes so that the trained model remained robust when deployed on real-world road footage.

VII. Conclusion and Future Work

We have successfully developed and deployed an automated pothole detection tool that combines YOLOv8 with OpenVINO optimisation and a Streamlit interface. The system achieves a strong balance between accuracy and computational efficiency, making it suitable for analysing road footage on standard CPU-based machines. Experimental results show that the method consistently detects potholes across varied road conditions and camera viewpoints, significantly reducing reliance on manual inspection.

Future work will focus on extending the offline analysis system into a real-time, on-board platform. Potential enhancements include integrating live camera feeds from moving vehicles, incorporating GPS modules for automatic geo-tagging of detected defects and adding support for multiple damage classes such as cracks and faded markings. We also plan to investigate model compression techniques such as quantisation and pruning to deploy the system on edge devices like Raspberry Pi or NVIDIA Jetson. Finally, large-scale field trials will be conducted with road maintenance authorities to evaluate usability and impact in real operational settings.

References

[1] J. Redmon, S. Divvala, R. Girshick and A. Farhadi, "You Only Look Once: Unified, Real-Time Object Detection," in *Proc. CVPR*, 2016.

[2] Ultralytics, "YOLOv8," GitHub repository, 2023. [Online]. Available: https://github.com/ultralytics/ultralytics

[3] Intel, "OpenVINO Toolkit," 2024. [Online]. Available: https://docs.openvino.ai

[4] IEEE, "POT-YOLO: Real-Time Road Potholes Detection Using Edge Segmentation-Based YOLOv8 Network," IEEE Xplore, 2023. [Online]. Available: https://ieeexplore.ieee.org/document/10542661.